SQL Joins: Conectando Dados

Entendendo Relacionamentos em Banco de Dados

Me. Sergio Souza Novak

28 de abril de 2026

O que é um JOIN?

- SQL Joins são usados para combinar linhas de duas ou mais tabelas.
- A combinação é feita com base em uma **coluna comum** (geralmente Chave Primária e Chave Estrangeira).
- É a base de qualquer sistema relacional: as informações são separadas em entidades, mas unidas na consulta.

Nossas Tabelas de Exemplo

Tabela: Alunos

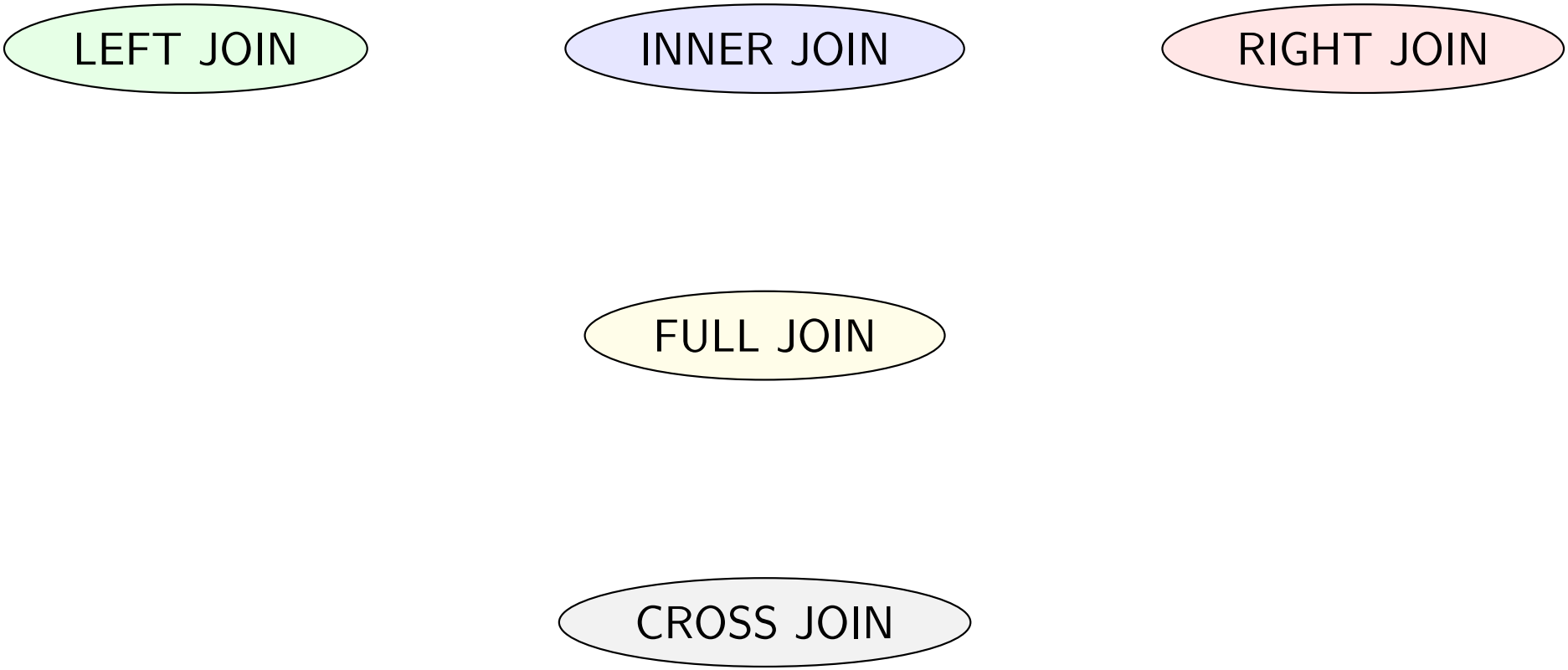
id	nome	c_id
1	Ana	10
2	João	20
3	Pedro	30

Tabela: Cursos

id	nome_curso
10	Engenharia
20	Direito
40	Medicina

Obs: Pedro tem curso 30 (não existe). Medicina não tem alunos.

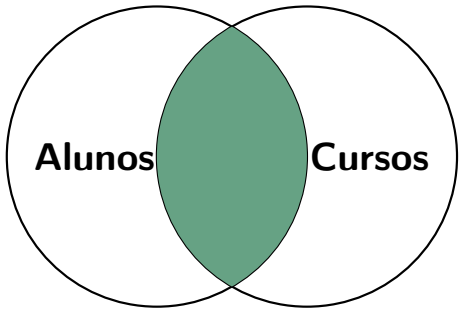
Tipos Principais de Joins



INNER JOIN (A interseção)

Retorna apenas registros que possuem **correspondência em ambas** as tabelas.

```
SELECT A.nome, C.nome_curso
FROM Alunos AS A
INNER JOIN Cursos AS C ON A.curso_id
    = C.id;
```



Resultado:

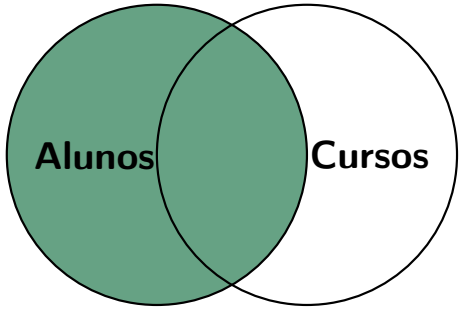
nome	nome_curso
Ana	Engenharia
João	Direito

** Pedro e Medicina ficaram de fora.*

LEFT JOIN (Prioridade à esquerda)

Retorna **todos** da tabela da esquerda e os correspondentes da direita.

```
SELECT A.nome, C.nome_curso
FROM Alunos AS A
LEFT JOIN Cursos AS C ON A.curso_id
    = C.id;
```



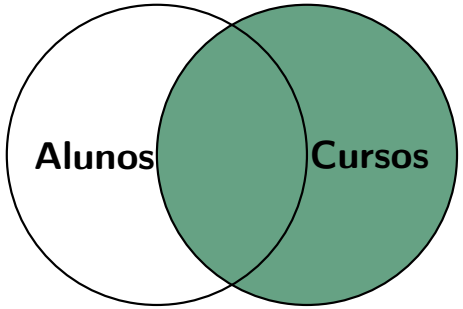
Resultado:

nome	nome_curso
Ana	Engenharia
João	Direito
Pedro	NULL

RIGHT JOIN (Prioridade à direita)

Retorna **todos** da tabela da direita e os correspondentes da esquerda.

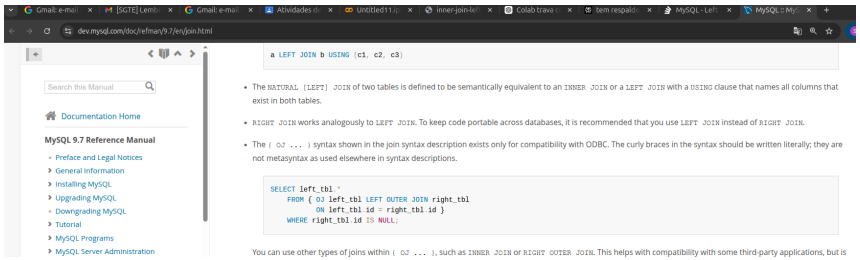
```
SELECT A.nome, C.nome_curso
FROM Alunos AS A
RIGHT JOIN Cursos AS C ON A.curso_id
    = C.id;
```



Resultado:

nome	nome_curso
Ana	Engenharia
João	Direito
NULL	Medicina

RIGHT JOIN: Implementação e Documentação



MySQL - Left Joins are Officially Preferred Over Right Joins?

Asked 13 years, 9 months ago Modified 13 years, 9 months ago Viewed 5k times

17

In the MySQL documentation for joins, a coworker pointed out this gem to me today:

> `RIGHT JOIN` works analogously to `LEFT JOIN`. To keep code portable across databases, it is recommended that you use `LEFT JOIN` instead of `RIGHT JOIN`.

- Algumas documentações (como as do MySQL) indicam que o `RIGHT JOIN` é internamente convertido para um `LEFT JOIN` com a ordem das tabelas invertida.
- Isso explica por que, na prática, muitos desenvolvedores preferem usar apenas `LEFT JOIN` para padronizar o código.

Compatibilidade: SQLite vs PostgreSQL

Nem todos os bancos de dados suportam todos os tipos de Join nativamente:

SQLite

Não implementa o `RIGHT JOIN` nem o `FULL JOIN`. É necessário inverter as tabelas em um `LEFT JOIN` para obter o mesmo resultado.

PostgreSQL

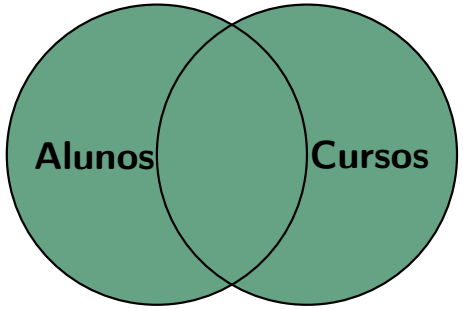
Implementação Completa. Suporta nativamente `RIGHT JOIN` e `FULL JOIN`, seguindo fielmente o padrão SQL ANSI.

Conclusão: O `LEFT JOIN` é o mais universal.

FULL OUTER JOIN (Tudo)

Retorna **todos** os registros de ambas as tabelas, unindo onde houver correspondência.

```
SELECT A.nome, C.nome_curso
FROM Alunos AS A
FULL JOIN Cursos AS C ON A.curso_id
    = C.id;
```



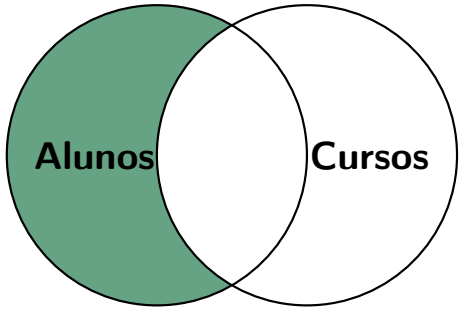
Resultado:

nome	nome_curso
Ana	Engenharia
João	Direito
Pedro	NULL
NULL	Medicina

LEFT JOIN (Exclusivo)

Retorna apenas os registros da esquerda que **não possuem par** na direita.

```
SELECT A.nome, C.nome_curso
FROM Alunos AS A
LEFT JOIN Cursos AS C ON A.curso_id
    = C.id
WHERE C.id IS NULL;
```



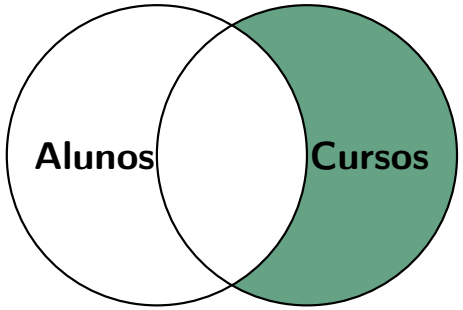
Resultado:

nome	nome_curso
Pedro	NULL

RIGHT JOIN (Exclusivo)

Retorna apenas os registros da direita que **não possuem par** na esquerda.

```
SELECT A.nome, C.nome_curso
FROM Alunos AS A
RIGHT JOIN Cursos AS C ON A.curso_id
    = C.id
WHERE A.id IS NULL;
```



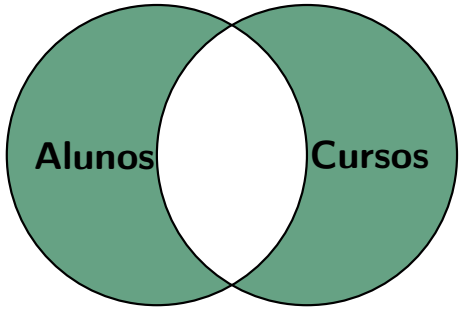
Resultado:

nome	nome_curso
NULL	Medicina

FULL OUTER JOIN (Exclusivo)

Retorna tudo o que **não é comum** entre as duas tabelas.

```
SELECT A.nome, C.nome_curso
FROM Alunos AS A
FULL JOIN Cursos AS C ON A.curso_id
    = C.id
WHERE A.id IS NULL OR C.id IS NULL;
```



Resultado:

nome	nome_curso
Pedro	NULL
NULL	Medicina

CROSS JOIN (Produto Cartesiano)

Combina **cada linha** da primeira tabela com **todas as linhas** da segunda.

```
SELECT A.nome, C.nome_curso FROM Alunos AS A
CROSS JOIN Cursos AS C;
```

Resultado (9 linhas):

Ana	Engenharia
Ana	Direito
Ana	Medicina
João	Engenharia... (continua)

Resumo para não esquecer

- **INNER JOIN**: O que elas têm em comum.
- **LEFT JOIN**: Tudo da esquerda + o que houver na direita.
- **RIGHT JOIN**: Tudo da direita + o que houver na esquerda.
- **FULL JOIN**: A união completa (mesmo sem par).
- **CROSS JOIN**: Todas as combinações possíveis.

Dúvidas?